

appendix C

Building knowledge graphs from structured sources

This appendix teaches you how to build your own knowledge graph (KG) from structured data sources. As we do in several places in the book, we focus on a bio-medical use case: here, the detection of microRNA disease associations. Figure C.1



Figure C.1 An example of the relationship between a disease (celiac disease) and microRNAs

shows the core of the KG we'll build. For this project, we will use CRISP-DM model introduced in chapter 2 (see figure C.2).

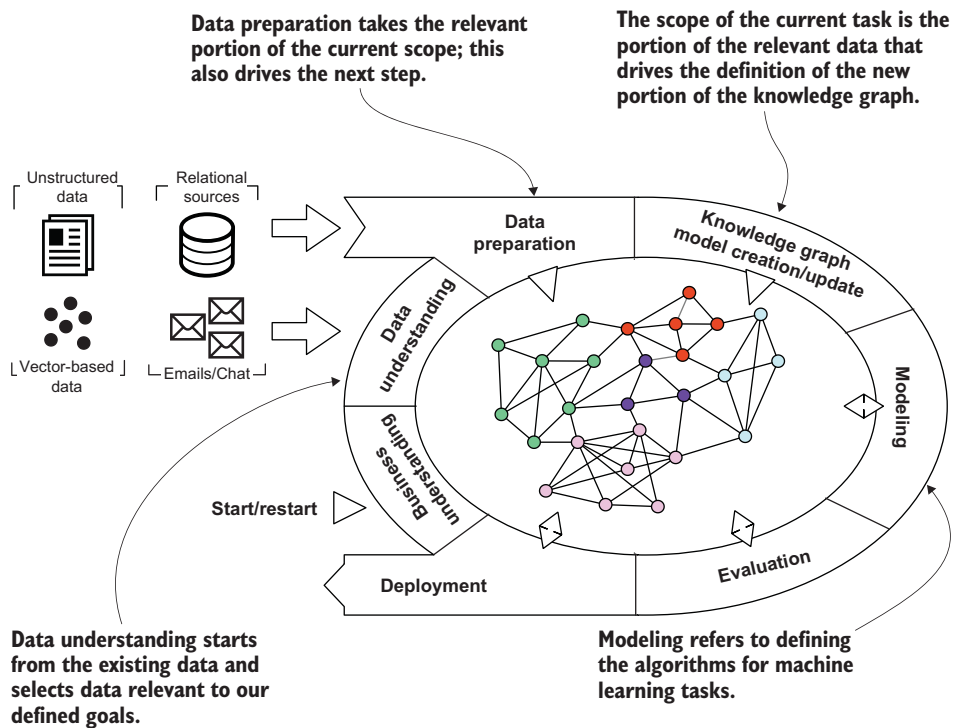


Figure C.2 CRISP-DM model adapted to KGs

C.1 *MicroRNA–disease association: Warmup*

MicroRNA–disease association is a very relevant use case for KGs in the biomedical space. In this section, we provide a brief biological description of what microRNAs are and how they are connected to diseases. We then outline the business goals and the data available to us for achieving those goals.

C.1.1 *Key concepts*

MicroRNA (hereafter referred to as miRNA) is a relatively newly discovered type of noncoding RNA (i.e., RNA that is not translated into a protein) [1]. These very small molecules (containing from 19 to 22 nucleotides) interfere with complementary messenger RNAs (mRNAs, which are supposed to be translated into a protein), causing what is called *gene silencing*: regulation of or interference with gene expression. miRNAs accomplish silencing through a combination of translational repression and mRNA destabilization [2]. Figure C.3 shows how normal encoding works, and figure C.4 illustrates how miRNAs affect it.

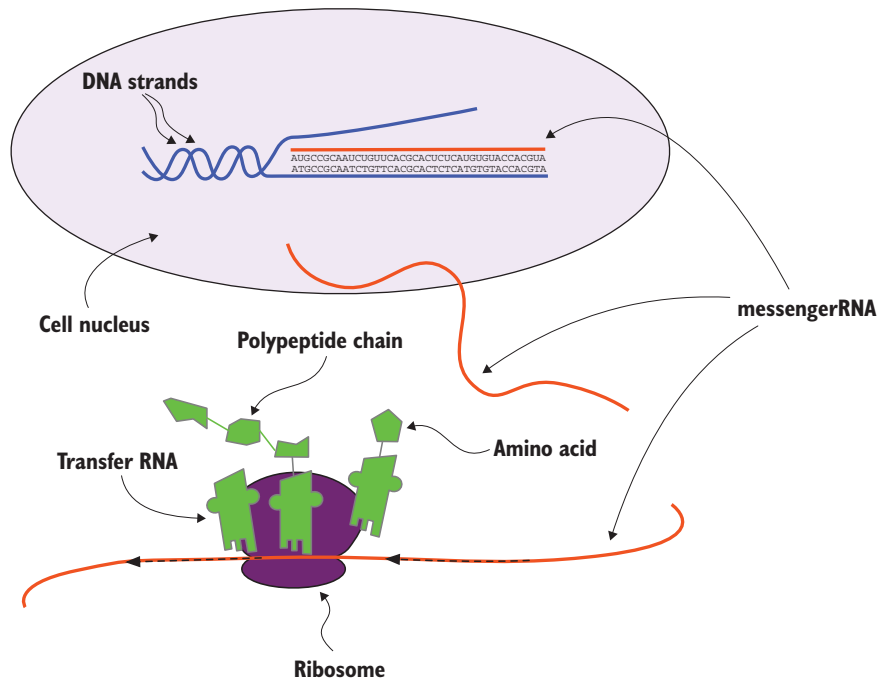


Figure C.3 Information encoded into DNA is transcribed into mRNAs and then translated into amino acid chains that make up the synthesized protein.

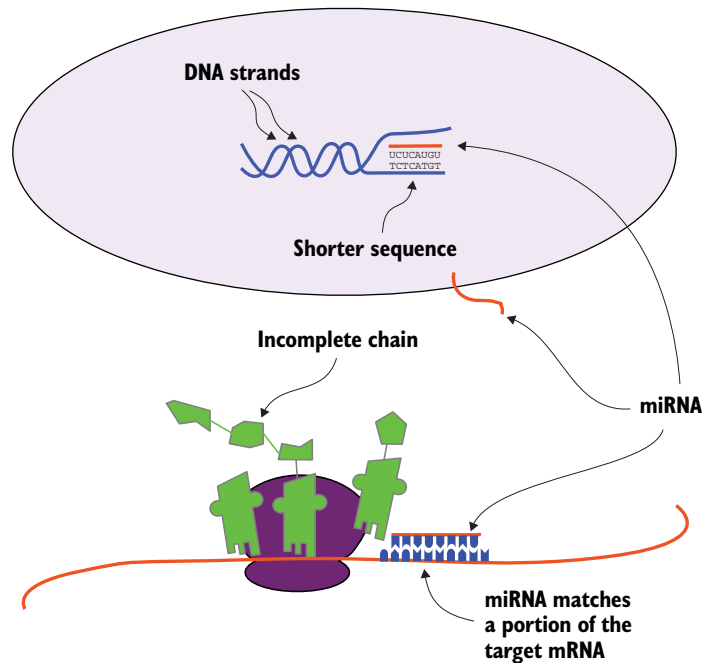


Figure C.4 miRNA sequences are too small to be translated into proteins but are large enough to target specific mRNAs. When this happens, the mRNA cannot flow through the ribosome, and protein synthesis cannot take place.

Studies have shown that miRNAs are involved in many important biological processes [3, 4], such as cell differentiation [5], proliferation [6], signal transduction [7], viral infection [8], and so on. Emerging evidence also implicates miRNAs in the pathogenesis of complex human diseases, such as cancer and metabolic disorders [9–15]; for example, researchers found that the mir-433 miRNA is involved in gastric carcinoma by regulating the expression of GRB2, which is a known tumor-associated protein [16]. This aspect is where this example will focus.

C.1.2 Business understanding

Suppose that we want to predict the connection between miRNAs and diseases. The potential number of combinations is huge: thousands of miRNAs have been identified, and their number and impact are likely more significant than previously suspected [17]. In addition, in vitro experiments to validate assumptions are expensive. The ability to predict correlations between miRNAs and diseases could help researchers narrow their investigations to the most likely ones [18], reducing the cost and time required to make such discoveries.

This scenario is related to the development of an intelligent advisor system (discussed throughout the book). The business goal also determines the type of analysis to be performed on the KG and the type of “advice” to be provided to end users. Here, our focus is on the construction phase; but understanding the required analysis helps refine the graph model, so it is an important consideration in the early stages of the process.

NOTE The approaches and techniques we demonstrate can be easily adapted to any scenario. Our goal is to show you how to construct a KG starting from existing structured data sources.

C.1.3 Data understanding

Although the research on miRNAs is relatively new, the amount of information available in this field of research is vast and easily accessible. A platform called Tools4miRs (<https://tools4mirs.org>) [19] claims to offer “all the tools you need to analyze your miRNAs.” It provides more than 170 methods and many databases.

We selected data sources relevant to our purposes. Each of the databases used is described and a link is provided, so you can check whether a new version is available. If this field interests you and you would like to extend your KG, we highly recommend using Tools4miRs as your entry point.

C.2 Building the miRNA knowledge graph

The KG we are going to build must capture the complexity of the connection between miRNAs and pathologies. This should make it possible to design a machine learning (ML) model that uses the abundant data and can learn to predict missing links (i.e., connections yet to be discovered). We don't cover the prediction algorithms here, but it's important to understand the opportunities that this type of KG construction will enable.

We could start by ingesting every available dataset related to miRNAs and diseases, and then let the ML model handle the analysis. This “greedy” approach assumes that

having more data means capturing a better understanding of the topic's complexity, which in turn should give the ML model more chances to discover the underlying rules that govern the relationship between miRNAs and disease.

Although this is true in theory, throwing everything into the KG is not a wise choice in practice:

- Every dataset has a signal-to-noise ratio. Even in good datasets, the noise can easily overwhelm the benefits of the signal if the dataset is only slightly related to link prediction.
- New datasets need to be reconciled to the existing KG before any graph ML algorithm can take the full advantage of the new information. Sometimes the reconciliation process is not straightforward, and the resulting relationships can be affected by errors, which can again amplify the noise associated with the new dataset.

We would rather begin by selecting datasets related to miRNAs that will serve our task: predicting pathology links. This approach will result in a KG with a relatively low degree of complexity, which is easier to reason about. The link prediction results from this KG can be used as a baseline, so it will be possible to quantify how new data sources can improve the model.

C.2.1 Importing known miRNA–disease connections

Let's start by identifying datasets that contain known connections between miRNAs and diseases. The data sources we selected for this first ingestion round are the Human miRNA Disease Database (HMDD; <http://www.cuilab.cn/hmdd>) [20–22], the Database of Differentially Expressed miRNAs in Human Cancers (dbDEMC; <https://www.biosino.org/dbDEMC>) [23], and miR2Disease (www.mir2disease.org) [24]. These datasets come from different sources and are the result of various research efforts; therefore, they encode the relations we are looking for in different ways and contain different associated information.

Each dataset will be processed and imported in a slightly different way, and they will be combined into a single graph. Figure C.5 illustrates how the merge will occur in practice with two toy datasets.

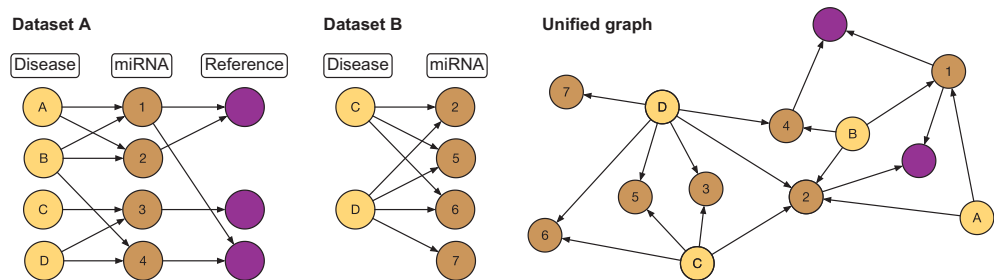


Figure C.5 Dataset A is more balanced and includes reference publications. Dataset B focuses on fewer diseases but includes a greater number of miRNAs. After ingestion, the information from both sources is unified in a single source of truth.

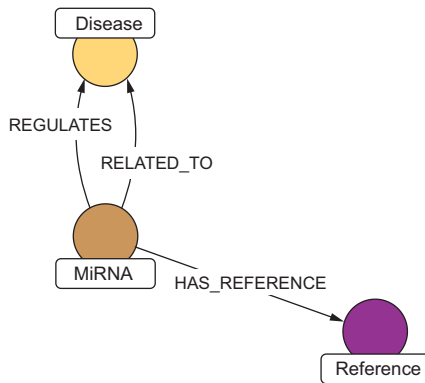


Figure C.6 First iteration target schema. It contains the type of relationships—**REGULATES** and **RELATED_TO**—that we would like to predict.

Considering the two sources, the information they contain, and our goal, the schema shown in figure C.6 suits our needs at this point. Here we are using two types of relationships, one for each of the two datasets we will be importing. This choice allows us to preserve the nuances of meaning that the two datasets give to the relationships. Additionally, we can quickly identify which of the two sources a relationship comes from when we look at the result of a query. Later, when we build our model, we can easily merge these two types of relationships if necessary.

We begin by importing the HMDD dataset. It contains experiment-supported evidence for human miRNA–disease associations.

This dataset will provide a solid foundation for our KG, as it is manually curated and captures the exact miRNA-to-disease link we want to predict. The following listing shows the implementation of the HMDD importer class that handles the data ingestion process.

Listing C.1 Importing the HMDD dataset

```

class HMDDImporter(BaseImporter):
    def __init__(self, argv):
        super().__init__(command=__file__, argv=argv)
        self._database = "hmd2.0"

    def get_rows(self, HMDD_file):
        with open(HMDD_file, 'r+', encoding="latin-1") as in_file:
            reader = csv.reader(in_file, delimiter='\t')
            header = next(reader)
            for row in reader:
                yield dict(zip(header, row))

    def import_HMDD(self, HMDD_file):
        query = """
            UNWIND $batch as item
            WITH
                trim(toLower(item.disease)) as disease,
                toLower(item.mir) as mir,item
            MERGE (d:Disease {name: disease})
            MERGE (m:MiRNA {name:mir})
            SET m:MiRNA_HMDD
            MERGE (m) - [r:RELATED_TO] -> (d)
        """

```

Extends BaseImporter (base dataset importing functionality)

Defines how to access raw data

Opens the textual data

Interprets the data as tab-separated values

Generates a dictionary-encoded representation of the records in the file

Defines how to ingest data in the graph

Preprocesses miRNA and disease names

Merges the Disease node using the name as identifier

Merges the relationship between the miRNA and Disease nodes

Reads the first line as a header

Unwinds a batch of records

Merges the miRNA node using the name as identifier

```

        SET r.description = item.description,
            r.pmid=item.pmid, r.category = item.category
        MERGE (ref:Reference {pubmed_id:item.pmid})
        MERGE (m)-[:HAS_REFERENCE]->(ref)
    """

    size = self.get_csv_size(HMDD_file, encoding="latin-1").
    self.batch_store(query, self.get_rows(HMDD_file),
                    size=size, strategy="aggregate")

    def set_constraints(self):
        with self._driver.session(database=self._database) as session:
            query = """
            CREATE CONSTRAINT ON (a:Disease) ASSERT a.name IS UNIQUE;
            CREATE CONSTRAINT ON (a:MiRNA) ASSERT a.name IS UNIQUE;

            CREATE CONSTRAINT ON (a:Reference)
                ASSERT a.pubmed_id IS UNIQUE;
            CREATE CONSTRAINT ON (a:Target) ASSERT a.name IS UNIQUE"""
            for q in query.split(";"):
                try:
                    session.run(q)
                except Neo4jClientError as e:
                    if (e.code !=
                        "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"):
                        raise e

[...]
```

Computes the number of records available →

Merges a Reference node using the PubMed ID as identifier and connects it to the miRNA node ←

Aggregates and stores the records ←

Enforces uniqueness for Disease, miRNA, Target, and Reference nodes →

Executes the query ←

Ignores errors ←

```

if __name__ == '__main__':
    importing = HMDDImporter(argv=sys.argv[1:])
    importing.set_constraints()
    importing.import_HMDD(HMDD_file)
    importing.close()

```

The next listing contains an example of a generated dictionary for an HMDD record.

Listing C.2 HMDD record sample

```

{'category': 'genetics_GWAS',
 'mir': 'hsa-mir-502',
 'disease': 'Carcinoma, Renal Cell, Clear-Cell',
 'pmid': 27346408,
 'description': "Polymorphism at the miR-502 binding site in the 3'[...] "}

```

This record indicates that the miRNA hsa-mir-502 is associated with kidney cancer, specifically clear cell renal cell carcinoma, and that this connection is supported by experiments described in the scientific publication identified by PubMed ID 27346408. We capture this information by

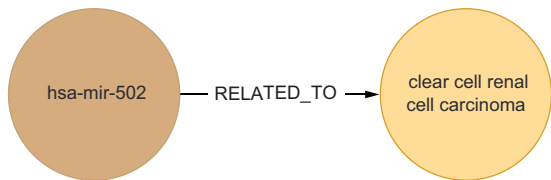


Figure C.7 Record as a graph portion stored in the database

creating the relevant miRNA and disease nodes and connecting them through the RELATED_TO relationships as shown in figure C.7.

After this import, we collected 1,207 distinct miRNAs, 18,732 distinct connections between these miRNAs, and 849 distinct diseases. This represents a solid foundation for our link prediction task.

The second dataset we import is the dbDEMC. It is an integrated database containing differentially expressed miRNAs focused on human cancer. It contains a collection of 403 miRNA expression datasets obtained by microarray platforms and miRNA sequencing. The following listing shows the implementation of the dbDEMC importer that processes this data.

Listing C.3 Importing the dbDEMC dataset

```
class DBDEMCImporter(BaseImporter):
    [...]
    @staticmethod
    def get_rows(miRNA_file):
        with open(miRNA_file, 'r+') as in_file:
            reader = csv.reader(in_file, delimiter='\t')
            header = next(reader)
            for row in reader:
                if len(row) < 2:
                    continue
                record = dict(zip(header, row))

                if record["Species"] != "Homo sapiens":
                    continue

                if len(record["CancerSubtype"]) > 1:
                    disease = record["CancerSubtype"].lower()
                else:
                    disease = record["CancerType"].lower()

                disease = (disease.replace(",", " ")
                           .replace("/", " ")
                           .replace("-", " "))

                name = record["miRNA_ID"].lower().strip()

                yield {
                    "name": name,
                    "disease": disease,
                    "experiment": record["ExperimentID"],
                    "regulated": record["Status"]
                }

    def import_dbDEMC(self, mirDB_file):
        exact_match_query = """
            UNWIND $batch as item
            MERGE (m:MiRNA {name: item.name})
            SET m:MiRNA_dbDEMC
            WITH m,item
            MERGE (n:Disease {name: item.disease})
```

Defines how to access the raw data

Filters out any incomplete or mistakenly parsed records

Keeps only human data

Selects the most specific description available for the current record

Selects the miRNA name

Generates a dictionary-encoded representation of the record

Defines how to ingest data in the graph

Unwinds a batch of records

Merges the miRNA node using the miRNA name as an identifier

Merges the Disease node using the disease name as identifier

Merges the relationship between the miRNA and the Disease nodes

```

SET n:DiseaseDbDEMC, n.name_in_db_demc = item.disease
MERGE (m)-[r:REGULATES {regulated: item.regulated}]->(n)
SET r.source = 'dbDEMC', r.experiment = item.experiment
"""
size = self.get_csv_size(miRDB_file)
self.batch_store(exact_match_query,
                 self.get_rows(miRDB_file), size=size)

```

Computes the number of records to ingest

Aggregates and stores the records

The next listing contains an example of a record generated for this dataset.

Listing C.4 dbDEMC record sample

```

{'name': 'hsa-miR-155',
 'disease': 'glioblastoma',
 'experiment': 'EXP00065',
 'regulated': 'UP'}

```

This record indicates that experiment EXP00065 shows elevated hsa-miR-155 miRNA levels in glioblastoma tumors compared with healthy adult brain tissue (this miRNA is upregulated in these tumor cells).

By using the `MERGE` clause—which doesn’t create new nodes if they already exist—we improve the richness of our knowledge base. We know from the HMDD dataset that hsa-mir-155 is significantly overexpressed in active multiple sclerosis lesions compared to controls. The same dataset also indicates that increased hsa-miR-155 levels are observed in ischemic cardiomyopathies (a disease affecting the heart muscle). At the end of the second ingestion (from dbDEMC), figure C.8 shows that the same node

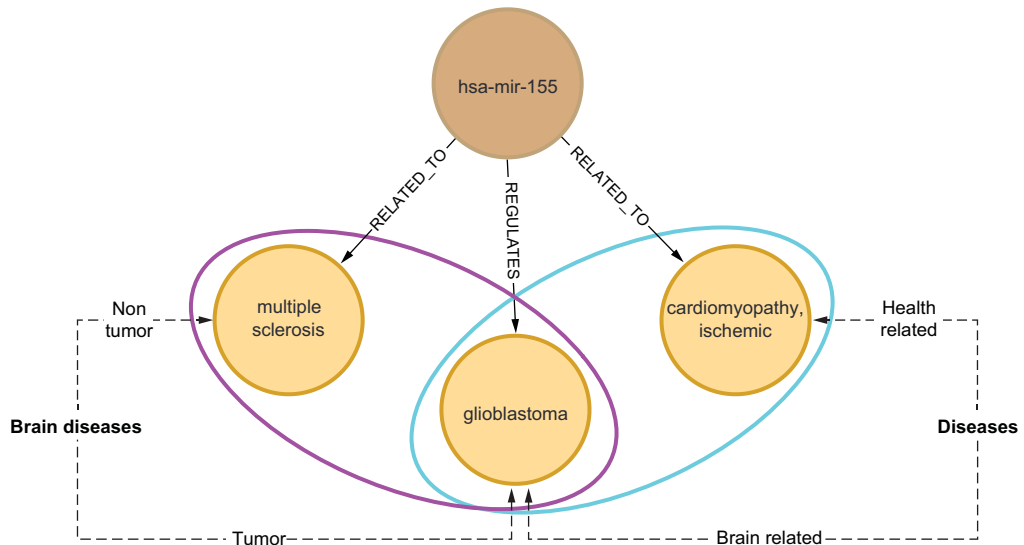


Figure C.8 Merging datasets allows information fusion. We are using two types of relationships for the two databases: `RELATED_TO` for the HMDD dataset and `REGULATES` for dbDEMC.

representing this specific miRNA is connected to both tumor and nontumor brain diseases (glioblastomas and multiple sclerosis, respectively), and it is also connected to brain and heart diseases (glioblastomas and ischemic cardiomyopathies, respectively).

Now, suppose that all three pathologies are somehow connected to excessive inflammation promoted by elevated levels of hsa-miR-155. Suppose also that another new pathology is correlated with the same inflammation process. In this case, the link prediction model has all the necessary information to infer a not (yet) documented relationship between the hsa-miR-155 miRNA and this new pathology.

We can enrich our KG further with a third dataset: the miR2Disease database. miR2Disease is a manually curated database that provides information about miRNA deregulation in various human diseases. It also provides a submission page that allows researchers to submit new miRNA–disease relationships, so we can expect it to grow over time. The following listing implements the miR2Disease importer that integrates this resource into our KG.

Listing C.5 Importing the miR2Disease dataset

```
class Mir2DiseaseImporter(BaseImporter):
    [...]
    def get_rows(self, miRDB_file):
        with open(miRDB_file, 'r+') as in_file:
            reader = csv.reader(in_file, delimiter='\t')
            for row in reader:
                if len(row) < 2:
                    continue
                yield {
                    "name": row[0].lower(),
                    "disease": row[1].lower(),
                    "regulated": row[2]
                }

    def import_miR2Disease(self, miR2Disease_file):
        query = """
            UNWIND $batch as item
            MERGE (d:Disease {name: item.disease})
            MERGE (m:MiRNA {name:item.name})
            MERGE (m) - [r:REGULATES] -> (d)
            SET r.regulation = item.regulated
        """
        size = self.get_csv_size(miR2Disease_file)
        self.batch_store(query, self.get_rows(miR2Disease_file),
                        size=size, strategy="aggregate").
```

Defines how to access the dataset's raw data

Filters out any incomplete or mistakenly parsed records

Generates a dictionary-encoded representation of the record

Defines how to ingest data

Unwinds a batch of records

Merges the Disease node using the name as identifier

Merges the miRNA node using the name as identifier

Merges the relationship between the miRNA and the Disease nodes

Aggregates and stores the records

Computes the number of records to ingest

Once these three datasets are imported, our KG comprises 4,874 distinct miRNAs, 118,806 distinct connections between these miRNAs, and 1,144 distinct diseases.

We can see how the unique miRNAs are distributed among the three datasets. The next listing shows the Cypher query that computes this distribution, allowing us to analyze the overlap and unique contributions of each data source.

Listing C.6 Computing the ingested miRNA distribution

```

MATCH (n:MiRNA)
WITH
  DISTINCT LABELS(n) AS labels,
  COUNT(*) as count
RETURN
  [l in labels where "MiRNA"<> l ] AS labels,
  Count
ORDER by count DESC

```

← All miRNA nodes have labels to track the dataset(s) of origin.

← Groups miRNA nodes depending on which dataset(s) they come from

← Removes "MiRNA" from the labels list because it provides no information about distribution

The result of the query looks like this:

	[MiRNA_dbDEMC]	2550
	[MiRNA_HMDD, MiRNA_dbDEMC]	583
	[MiRNA_HMDD, MiRNA_dbDEMC, MiRNA_miR2Disease]	328
	[MiRNA_HMDD]	280
	[MiRNA_dbDEMC, MiRNA_miR2Disease]	84
	[MiRNA_miR2Disease]	32
	[MiRNA_HMDD, MiRNA_miR2Disease]	15

Figure C.9 shows these results as a Venn diagram. There is a fairly balanced distribution between shared and nonshared miRNAs. This is relevant because for shared miRNAs, our KG and subsequent ML tasks will benefit from knowledge derived from multiple datasets. Nonshared miRNAs represent the unique contribution of each dataset we have ingested.

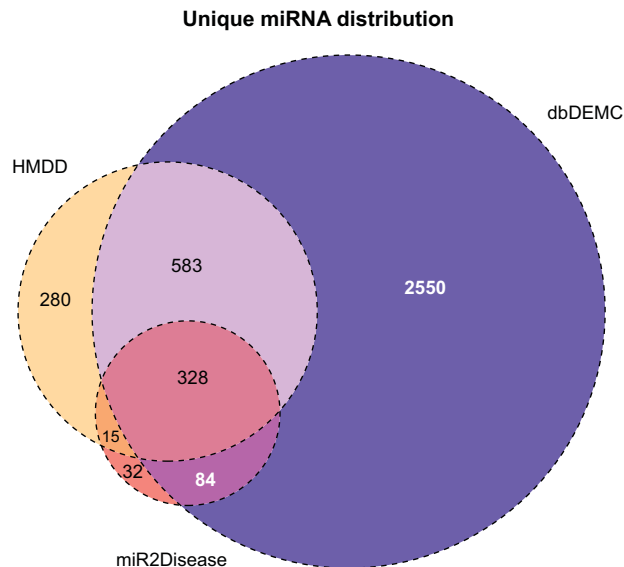


Figure C.9 Venn diagram illustrating the unique distribution of miRNAs. It shows the overlapping miRNAs among the datasets.

C.2.2 Importing the disease ontology

Looking closely at the imported data, we notice that data sources use different terms to refer to the same disease. This is a common problem, as different datasets, even those related to the same topic, often use different standards in defining objects. This is particularly frequent in the case of biological and medical datasets. A detrimental effect of this misalignment is that, for example, two miRNAs may be connected to two apparently different diseases, when in fact they refer to the same disease with different names.

Figure C.10 shows an example of this problem. In this case, each dataset referred to Burkitt lymphoma with different wording, relying on different disease naming conventions and thus resulting in three different disease nodes.

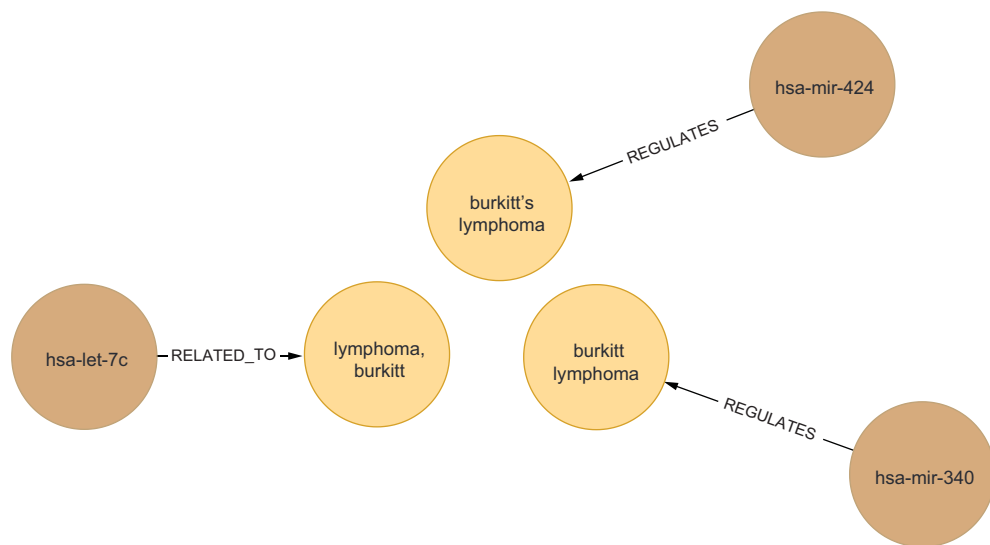


Figure C.10 These miRNAs appear to refer to different diseases.

As a consequence, the KG cannot be considered correct. As you know, the KG is a representation of real-world entities. If different entities represent the same concept, the graph ceases to be a reliable source of truth. Furthermore, when we perform our link prediction task, the model may be misled into believing that the cited miRNAs are disconnected, preventing it from learning properly. In general, when this type of misalignment happens, every representation we build on top of it will be proportionally deteriorated.

DISEASE NORMALIZATION WITH UMLS AND SCISPACY

Luckily, many ontologies are available that we can use to “normalize” the nomenclature of diseases from different datasets. Here we will use the Unified Medical Language System (UMLS; <https://www.nlm.nih.gov/research/umls>) [25] ontology. The

UMLS integrates and distributes key terminology, classification, and coding standards, along with associated resources, to promote the creation of more effective and interoperable biomedical information systems and services, including electronic health records.

We will use `scispaCy` (<https://allenai.github.io/scispaCy>), a Python package containing `spaCy` models for processing biomedical, scientific, or clinical text. `scispaCy` can perform automatic named entity recognition of UMLS entities, returning the canonical name, concept ID, and type ID for every identified entity in the disease name property. We can use it to automatically infer the canonical name for every disease node we ingested, generating new `NormalizedDisease` nodes that we will connect to equivalent `Disease` nodes (see figure C.11). We decided to add a new node to our schema rather than merging the three nodes into a single node because we want to retain the original structure. This is useful for two reasons:

first, it will be easier to review the results and correct any errors; and second, resetting everything and rerunning, if necessary, will be much more straightforward.

The following listing implements the disease normalization process, using natural language processing (NLP) techniques to standardize disease entities across our different data sources.

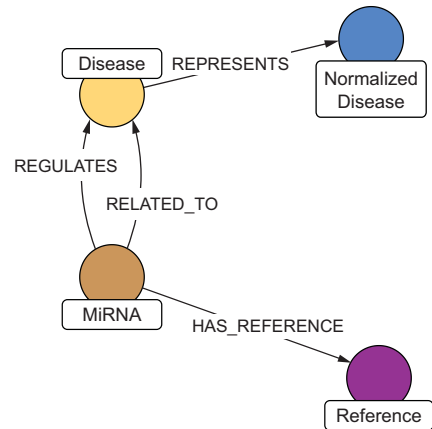


Figure C.11 Updating the target schema containing the `NormalizedDisease` node

Listing C.7 Normalizing diseases

```

class Reconciliator(BaseImporter):
    def __init__(self, argv):
        super().__init__(command=__file__, argv=argv)
        self._database = "hmd2.0"
        self.resolver = DiseaseResolver()

    def get_normalized_diseases(self):
        with self._driver.session(database=self._database) as session:
            diseases_data = session.run("""
                MATCH (d:Disease)
                RETURN id(d) as id, d.name as name""").data()

            diseases_text = [d["name"] for d in diseases_data]
            disease_ids = [d["id"] for d in diseases_data]

            diseases_text = [
                ".join(i for i in reversed(d.split(",))).strip()
                for d in diseases_text]
  
```

Reuses functionality from `BaseImporter`

Defines how to access the dataset's raw data

Fetches diseases to be normalized from Neo4j

Extracts the disease name

Extract the Disease node ID

Converts items like "leukemia, lymphocytic, chronic, b-cell" into "b-cell chronic lymphocytic leukemia"

```

diseases_items = [self.resolver.nlp(disease)
                  for disease in diseases_text]

disease_normalized = [
    self.resolver.normalize(item)
    for item in diseases_items]

diseases = [{
    "source_id": disease_id,
    "name": disease_name,
    "umnl_id": disease_UMNLS_ID}
    for disease_id, (disease_name, disease_UMNLS_ID)
    in zip(disease_ids, disease_normalized)]

return diseases

def import_normalized_diseases(self):
    query = """
        UNWIND $batch as item
        MATCH (d:Disease)
        WHERE id(d) = item.source_id
        MERGE (nd:NormalizedDisease {name:item.name})
        SET nd.umnl_id = item.umnl_id
        MERGE (d)-[:REPRESENTS]->(nd)
    """
    diseases = self.get_normalized_diseases()
    self.batch_store(query, iter(diseases),
                    size=len(diseases), strategy="aggregate") #0

```

Runs the NLP pipeline for every disease name text and returns an object containing the detected medical entities

Normalizes the disease using information from the scispaCy pipeline

Converts normalized diseases into a dictionary

Defines how to ingest data

Unwinds a batch of records

Selects the original Disease node

Creates a node representing the normalized disease if it does not exist

Links the original disease with its normalized version

Aggregates and stores records

The next listing shows the resolution logic, which is encapsulated in DiseaseResolver.

Listing C.8 DiseaseResolver class

```

class DiseaseResolver:
    full = ["Finding", "Organ or Tissue Function", "Tissue"]
    banned = ["Human", "Body Part, Organ, or Organ Component",
              "Qualitative Concept", "Temporal Concept",
              "Functional Concept", "Body Space or Junction",
              "Spatial Concept"]

    def __init__(self):
        self.nlp = nlp = spacy.load("en_core_sci_sm")
        config = {
            "resolve_abbreviations": True,
            "linker_name": "umls"}
        nlp.add_pipe("scispacy_linker", config=config)
        linker = nlp.get_pipe("scispacy_linker")
        self.type_tree = linker.kb.semantic_type_tree.
        self.cui_to_entity = linker.kb.cui_to_entity

```

Defines a set of entity types that must be matched in full to be considered a disease

Defines a set of entity types that can't be considered valid diseases

Creates a scispaCy NLP model

Fetches the linker

Sets up the linker, which detects UMLS entities

Fetches the mapper from the UMLS entity to the Concept ID used as an index by the UMLS ontology

Fetches the semantic tree of types from UMLS to label entities (e.g., "multiple sclerosis" is labeled "Disease or Syndrome")

```

def canonical(self, entity):
    """get canonical name from entity"""
    entities = entity._.kb_ents
    if len(entities) == 0:
        return
    # select the first entity
    return self.cui_to_entity[entities[0][0]].canonical_name

def types(self, entity):
    """return semantic types for the entity"""
    entities = entity._.kb_ents
    if len(entities) == 0:
        return []
    return [self.type_tree.get_canonical_name(t)
            for t in self.cui_to_entity[entities[0][0]].types]

@staticmethod
def matchesAll(entity):
    """return true if the entity covers the whole content"""
    return entity.start == 0 and entity.end == len(entity.doc)

def containsOnly(self, entity, targets):
    """return true if the entity types are
       within the target types"""
    intersection = set(self.types(entity)).intersection(targets)
    return (intersection == set(self.types(entity)))

def validEntity(self, entity):
    """ exploits the entity types to detect if an entity is
       correctly identified as disease """
    if self.containsOnly(entity, self.banned):
        return False
    if self.containsOnly(entity, self.full):
        return self.matchesAll(entity)
    return True

def normalize(self, item):
    """main entrypoint: convert item into a normalized disease
       return ( normalized_name, UMLS_ID | None )
    """
    if len(item.ents) == 1:
        return self.normalize_entity(item)
    if len(item.ents) > 1:
        return self.normalize_default(item)
    return self.normalize_default(item)

def normalize_entity(self, item):
    """ normalize item when there is only one detected entity """
    entity = item.ents[0]
    if self.validEntity(entity):
        return self.canonical(entity), entity._.kb_ents[0][0]
    return self.normalize_default(item)

```

Gets the assigned canonical name for an entity

Gets the types associated with an entity

Checks whether the entity spans the entire text

Checks whether an entity contains only a specific set of type labels

Checks whether an entity can be considered a disease

If the type labels are among those defined earlier that never apply to disease, the model failed to recognize the disease.

If the type labels are among those defined earlier which can represent a disease only if the entity spans the entire text, and that is not the case, the model failed to recognize the disease as a whole.

The type labels represent valid diseases.

If only a single entity is found, we use the normalization logic defined in `normalize_entity`.

If more than one entity is found for the current disease, we use the default normalization.

Defines the logic to normalize a disease if only one entity is found

If the detected entity is not a valid disease, we use the default normalization.

If the detected entity is a valid disease, we return the tuple (<canonical name>, <UMLS Concept Id>) extracted from the entity metadata.

```
def normalize_default(self, item):
    """When no other better options are available
       return capitalized version of disease"""
    item = str(item)
    item = " ".join(i.capitalize() for i in item.split())
    return item.strip(), None
```

Defines the default logic to normalize a disease

Returns a capitalized version of the disease text and None as ID

After this normalization step, the three different Burkitt lymphoma nodes with their varying spellings are connected by a single normalized *Burkitt Lymphoma* node (see figure C.12). We have effectively reduced graph fragmentation by adding an extra node that connects previously disconnected components. We can evaluate the graph connectivity—with and without the normalization nodes—using the weakly connected component (WCC) algorithm that we use elsewhere in the book. With WCC, we can detect sets of connected nodes that form a single connected component: in other words, we can identify the number of disconnected subgraphs in our graph, labeling each node according to the disconnected subgraph to which it belongs.

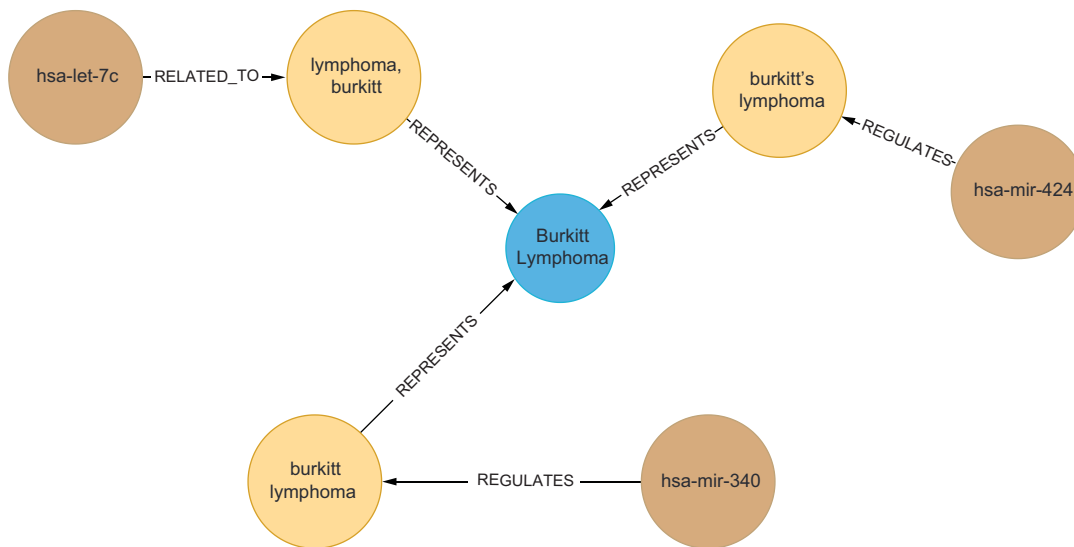


Figure C.12 miRNAs are now connected to the NormalizedDisease node *Burkitt Lymphoma*.

EVALUATING THE EFFECT OF NORMALIZATION

Through our normalization, we may have connected previously disconnected graphs. We can measure this by running WCC before and after the normalization process and comparing the results.

Before running WCC, as with any other GDS algorithm, we must create a named in-memory representation of the graph we are going to analyze. In this case, we will

create two representations: one with the `NormalizedDisease` nodes and their relative relationships, and one without. The following listing shows the Cypher queries that project these two graph representations into memory, setting up our environment for comparative analysis.

Listing C.9 Creating in-memory graphs

```
CALL gds.graph.project(
  "not-normalized",
  ["MiRNA", "Disease"],
  ["REGULATES", "RELATED_TO"]);
```

Creates an in-memory graph
Name of the non-normalized version of the graph
Selects miRNA and Disease node labels

```
CALL gds.graph.project(
  "normalized",
  ["MiRNA", "Disease", "NormalizedDisease"],
  ["REGULATES", "RELATED_TO", "REPRESENTS"]);
```

Name of the normalized version of the graph
Selects miRNA, Disease, and NormalizedDisease nodes
Selects the relationships between miRNA, Disease, and NormalizedDisease
Selects relationships between miRNA and Disease

We can now run the WCC algorithm once over both in-memory representations and compare the results. The next listing executes the WCC algorithm on the non-normalized graph.

Listing C.10 Running WCC over the non-normalized graph

```
CALL gds.wcc.stream('not-normalized')
YIELD nodeId, componentId
RETURN componentId AS subgraph, count(nodeId) AS componentSize
```

Returns the ID of each node and of the subgraph it belongs to
Computes the size distribution of each component (subgraph)
Calls the WCC algorithm and returns the results without modifying the in-memory representation

Table C.1 shows the distribution of components before normalization, revealing the baseline connectivity patterns that we'll compare against after our disease entity standardization. After examining the connectivity of the non-normalized graph, we can perform the same analysis on our normalized representation to assess the impact of disease standardization (see listing C.11).

Table C.1 WCC components distribution before normalization

subgraph	componentSize
0	5010
1166	3
1838	2

Listing C.11 Running WCC over the normalized graph

```

CALL gds.wcc.stream('normalized')
YIELD nodeId,componentId
RETURN componentId AS Subgraph, count(nodeId) AS ComponentSize

```

Returns the ID of each node and of the subgraph it belongs to
 Calls the WCC algorithm and returns the results without modifying the in-memory representation
 Computes the size distribution of each component (subgraph)

Table C.2 shows the components distribution after normalization, enabling a direct comparison with the prenormalization state to quantify the improvements in graph cohesion. In this case, there are no significant changes when comparing the graph structures before and after normalization, because we have only a single, large, connected component containing almost every node.

Table C.2 WCC components distribution after normalization

subgraph	componentSize
0	6033
1166	4
1838	3

Although having a well-connected graph with negligible fragmentation is good news for most graph applications, here we have to apply other techniques to quantify the impact of the normalization step. The GDS library contains several community detection algorithms that can statistically evaluate the structural changes from the normalization process. However, these changes are more difficult to interpret when we use other community detection algorithms than when we use WCC.

Let's consider the subgraph shown in figure C.13. Here *hsa-mir-199a** is connected to *hsa-mir-182* through *hepatocellular carcinoma (hcc)*, and at the same time, *hsa-mir-182* is connected to *hsa-mir-4728* through *carcinoma, papillary, thyroid*. We can say that the distance between *hsa-mir-199a** and *hsa-mir-182* is equal to 1, because the shortest path connecting the two miRNAs contains only one Disease node. The distance between *hsa-mir-199a** and *hsa-mir-4728* is equal to 2, because the shortest path passes through two Disease nodes. However, we now know that *burkitt's lymphoma* and *lymphoma, burkitt* are actually the same disease; thus, the distance between *hsa-mir-199a** and *hsa-mir-4728* should be 1.

Figure C.14 illustrates the same chain of miRNAs after disease normalization. The previously disparate disease nodes are now connected through the shared *NormalizedDisease* node, effectively reducing the path lengths between related miRNAs.

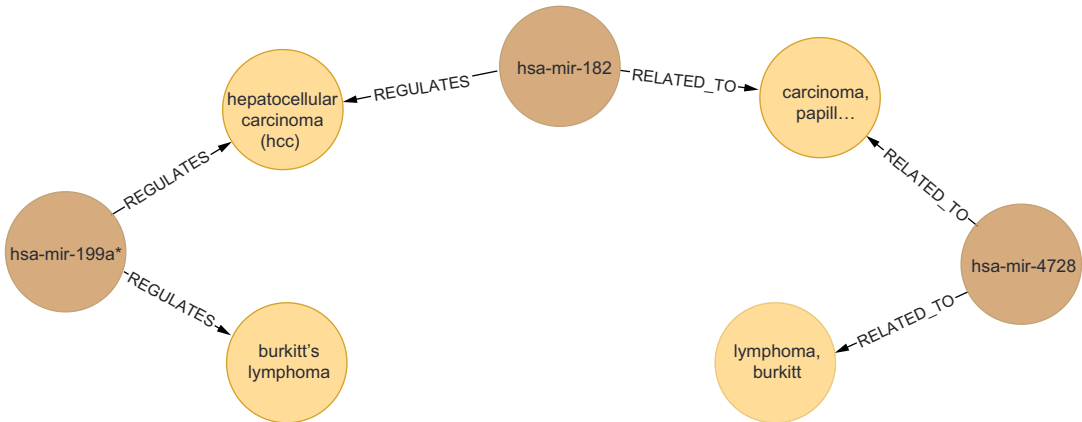


Figure C.13 Chain of miRNAs connected through Disease nodes

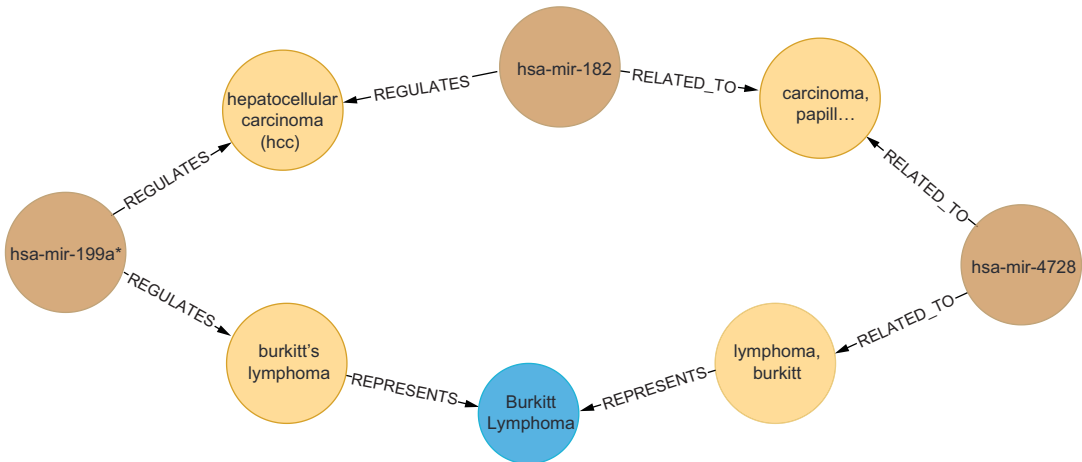


Figure C.14 The same chain of miRNAs as in figure C.13 after the connection through a `NormalizedDisease` node

In general, we expect the distances between miRNAs to be shorter after disease normalization. We can measure these distances using the *all-pairs shortest path* (APSP) algorithm, which at the time of writing is available in the alpha tier in the Path Finding algorithms of the Neo4j GDS library.

We will first create an in-memory graph representation where two miRNA nodes are connected only if there exists at least one `Disease` node connected to both of them. This projection will represent the state of the graph before the normalization step.

We will also create a second in-memory graph representation where two miRNAs' nodes are connected if there exists a chain (Disease)-[]-(NormalizedDisease)-[]-(Disease) connecting them. This projection will represent the state of the graph after the normalization step. The following Cypher query creates this in-memory graph connecting miRNAs through their shared Disease nodes.

Listing C.12 miRNA-to-miRNA connection through Disease nodes

```
call gds.graph.project.cypher(
  "DiseaseDistance",
  "MATCH (n:MiRNA) return id(n) as id",
  "MATCH (a:MiRNA) -[:REGULATES|RELATED_TO]->
    (:Disease) <-[:REGULATES|RELATED_TO]- (b:MiRNA)
  WHERE id(a)<id(b)
  RETURN distinct #F
    id(a) as source,
    id(b) as target")
```

Creates the graph

Names the graph

Selects all the miRNA nodes

Generates the relationship: a and b are connected if they are connected through one Disease.

Ignores b -> a if a -> b already exists

Returns the node IDs as source and destination to generate the relationship in the graph representation

a and b may be connected by multiple diseases, but we just need to know whether a link exists.

The next listing generates an in-memory graph that uses the NormalizedDisease nodes created to normalize diseases.

Listing C.13 miRNA-to-miRNA connection through NormalizedDisease nodes

```
call gds.graph.project.cypher(
  "NormalizedDiseaseDistance",
  "MATCH (n:MiRNA) return id(n) as id",
  "MATCH p1=(a:MiRNA) -[:REGULATES|RELATED_TO]->() -[:REPRESENTS]->(d)
  MATCH p2=(d) <-[:REPRESENTS]-() <-[:REGULATES|RELATED_TO]- (b:MiRNA)
  WHERE id(a)<id(b)
  RETURN distinct #F
    id(a) as source,
    id(b) as target")
```

Selects all the miRNA nodes

Creates the graph

Names the graph

Ignores b -> a if a -> b already exists

Returns the node IDs as source and destination to generate the relationship in the graph

Generates the relationship: a and b are connected if they are connected through one NormalizedDisease.

a and b may be connected by multiple NormalizedDiseases, but we just need to know whether a link exists.

These queries may require some time to execute. In general, calls to `gds.graph.project` are faster than calls to `gds.graph.project.cypher`, because the former uses information about graph nodes and edges that is already stored in the database. On the other hand, Cypher projections are more flexible and more useful for exploratory and debugging purposes because we can project one graph onto another using computations, similar to what we just did.

Listing C.14 computes the distance before normalization, and listing C.15 computes the distance after normalization. The results of both listings are summarized in table C.3.

Listing C.14 Running APSP on a prenormalization graph

```
CALL gds.allShortestPaths.stream('DiseaseDistance', {})
YIELD distance
RETURN distinct distance, count(distance) AS Count
```

← Returns the distance distribution

← Executes the undirected APSP and returns the results without modifying the in-memory representation

Listing C.15 Running APSP on a post-normalization graph

```
CALL gds.allShortestPaths.stream('NormalizedDiseaseDistance', {})
YIELD distance
RETURN distinct distance, count(distance) AS Count #B
```

← Returns the distance distribution

← Executes the undirected APSP and returns the results without modifying the in-memory representation

Table C.3 Distance distributions

Distance	Path count before	Path count after	Variation
1	5,911,305	6,179,329	+4.5%
2	1,244,305	1,010,851	-18.7%
3	35,795	25,888	-27.6%
4	870	612	-29.6%
5	46	22	-52.1%
6	1	0	-100%

We can see that the count of miRNA pairs at a distance of 1 has increased by about 5%, which means the distance for these pairs is shortened. The decrease in the count for the other distances also suggests a shift toward shorter distances.

This is a significant analysis and a notable achievement. Many of the embeddings' techniques are based on message passing, which means they use relationships to pass messages that are used to compute new embeddings at each iteration. Shorter connections among relevant nodes relate directly to higher-quality final embeddings.

Using LLMs for entity normalization

Although our approach using *scispaCy* successfully normalized disease entities like *burkitt lymphoma*, *lymphoma*, *Burkitt*, and *Burkitt's lymphoma*, LLMs could provide an alternative solution in situations where specialized biomedical NLP tools are unavailable or unsuitable. LLMs benefit from their exposure to vast corpora of biomedical literature during pretraining, giving them an inherent understanding of terminology variations.

LLMs could potentially address entity reconciliation challenges in several ways:

- *Handling terminology variations*—LLMs can recognize semantic equivalence between terms even when they don't follow predictable transformation patterns: for example, recognizing that “Gastric adenocarcinoma” and “Stomach cancer” refer to the same entity despite using different words.
- *Domain-agnostic application*—The entity reconciliation approach demonstrated here isn't limited to biomedical domains. LLMs could provide similar normalization capabilities in various domains where specialized tools like *scispaCy* aren't available, such as legal documents, financial reports, or technical specifications.
- *Zero-shot capabilities*—Unlike our current approach, which requires specific ontologies like the UMLS, LLMs might perform reasonably well for entity normalization without requiring external knowledge bases, particularly for common entities.

Despite these potential benefits, key limitations should be considered. The probabilistic nature of LLM output may lead to inconsistent entity mappings across different runs, potentially compromising the reproducibility of the KG construction process. Moreover, deploying LLMs for entity reconciliation at scale would require substantial computational resources compared to more lightweight approaches, such as *scispaCy*.

C.2.3 Importing miRNA information

So far, we have ingested datasets containing known miRNA-to-disease associations and improved the quality of the disease relationships through disease normalization and merging. Both steps help establish rich, high-quality relationships, providing a solid foundation for the link prediction task. We can further enrich the KG by ingesting additional datasets that provide information about miRNAs and their connections.

As shown in the updated schema in figure C.15, some of the new relationships connect miRNAs to each other, providing direct information about miRNA similarities and relations (direct similarity). Others, such as miRNA to Target and miRNA to Reference, provide indirect connection information (indirect similarity). If two miRNAs bind to the same target mRNA, for example, they are similar in the sense that they are regulating—or silencing—the same gene expression. Similarly, two or more miRNAs cited in the same publication indicate that they are somehow related, at least from the authors' perspective. Figure C.16 illustrates this idea with an example dataset.

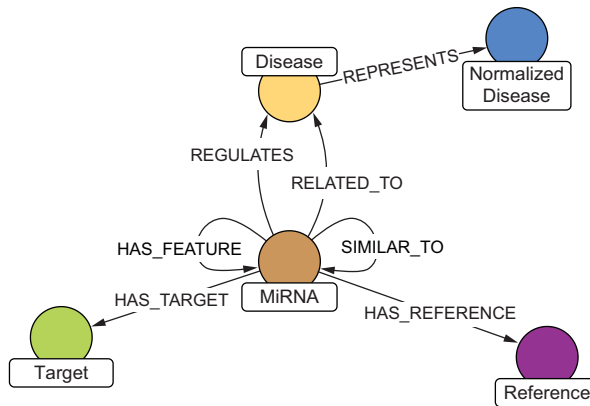


Figure C.15 Updated version of the target schema

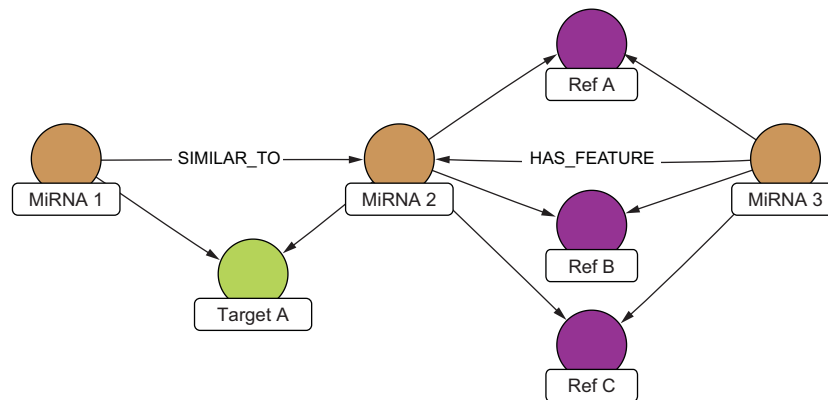


Figure C.16 Example of direct similarity and similarity induced by shared nodes. The model will learn which similarity is most relevant for the link prediction task.

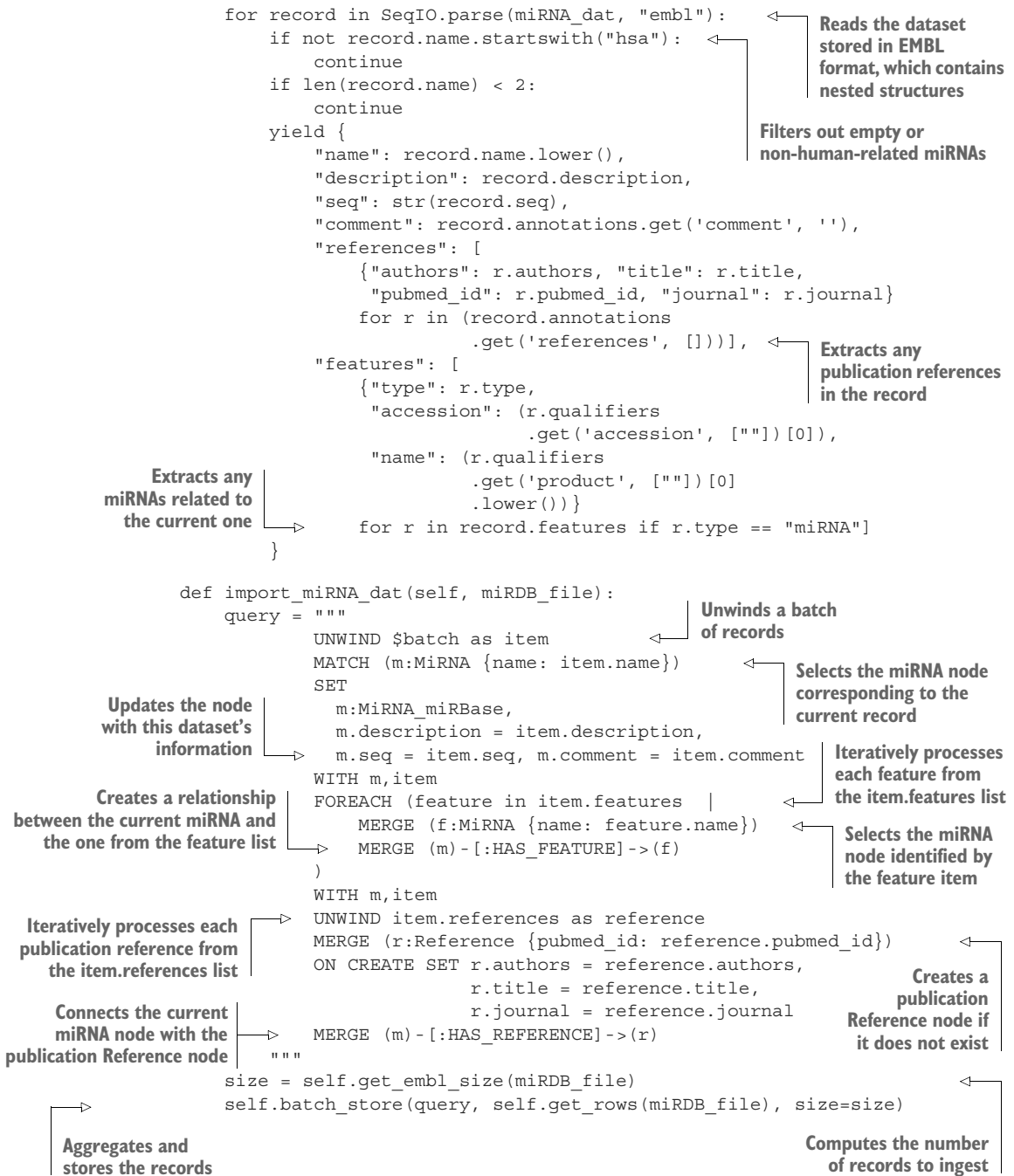
Let's start by importing the miRBase dataset (www.mirbase.org) [26–29], a searchable database of about 200 published miRNA sequences and annotations. For each miRNA, the dataset reports a list of relevant publications mentioning it as well as a list of connected miRNAs. The following listing implements the miRBase importer that handles the extraction and integration of this miRNA reference database into our KG.

Listing C.16 Importing the miRBase dataset

```
from Bio import SeqIO

class BioImporter(BaseImporter):
    [...]
    @staticmethod
    def get_rows(miRNA_dat):
```

Defines how to access
the dataset's raw data



The following listing is an example of the records provided by miRBase. It contains a wealth of relevant information that allows us to connect miRNAs to one another and to reference articles.

Listing C.17 miRBase record sample

```

name: hsa-let-7a-1
description: Homo sapiens let-7a-1 stem-loop
comment:
    let-7a-3p cloned in [6] has a 1 nt 3' extension (U), which is
    incompatible with the genome sequence.
seq:
    UGGGAUGAGGUAGUAGGUUGUAUAGUUUUAGGGUCACACCCACCACUGGGAGAUAAACU
    AUACAAUCUACUGUCUUUCCUA
features:
    - accession: MIMAT0000062
      name: hsa-let-7a-5p
      type: miRNA
    - accession: MIMAT0004481
      name: hsa-let-7a-3p
      type: miRNA
references:
    - authors: Lagos-Quintana M, Rauhut R, Lendeckel W, Tuschl T
      journal: Science. 294:853-858(2001).
      pubmed_id: 11679670
      title:
          Identification of novel genes coding for small expressed RNAs
    - authors:
        Suh MR, Lee Y, Kim JY, Kim SK, Moon SH, Lee JY, Cha KY,
        Chung HM, Yoon HS, Moon SY, Kim VN, Kim KS
      journal: Dev Biol. 270:488-498(2004).
      pubmed_id: 15183728
      title:
          Human embryonic stem cells express a unique set of miRNAs
    [...]

```

The next dataset to be ingested is miRDB (<https://mirdb.org>) [30, 31], an online database for miRNA target prediction and functional annotations. As previously discussed, miRNAs primarily function by downregulating the expression of their target genes. Thus, accurately predicting miRNA targets is critical for characterizing miRNA functions. The targets in the dataset are predicted using a bioinformatics tool, miR-TargetLink 2.0 (<https://ccb-compute.cs.uni-saarland.de/mirtargetlink2>) [32], which was developed by analyzing miRNA–target interactions from high-throughput sequencing experiments. miRDB includes 3.5 million predicted targets regulated by 7,000 miRNAs in 5 species; however, we will focus on the miRNAs we have already imported into our graph (i.e., those related to humans).

The next listing shows a miRDB record sample including the confidence score, which helps evaluate the strength of the miRNA–target association. The result of the import is shown in figure C.17.

Listing C.18 miRDB record sample, including confidence score

```

{'name': 'hsa-mir-96-5p',
 'target': 'NM_012214',
 'value': 90.3926}

```


C.3 Exploring and analyzing the miRNA KG

Let's review the graph we've constructed, extract some information, and validate the quality of the knowledge it contains before moving on to more complex tasks. To do so, we can run a query that allows us to observe similarities between nodes of the same type and nodes of different types. These kinds of similarities represent implicit relationships that ML algorithms will use to perform their tasks. As we will see, during the training phase, many embeddings' algorithms learn to identify which implicit relationships are more useful to obtain the desired results.

In case you didn't create the database, you can import ours (<https://downloads.graphaware.com/neo4j-db-seeds/hmdd2.0.backup>). You can also use this backup to verify your database if you run the full process. Add the following line to the neo4j.conf file:

```
dbms.databases.seed_from_uri_providers=URLConnectionSeedProvider
```

Then run the following command.

Listing C.20 Importing the miRNA database from a backup

```
CREATE DATABASE `hmdd2.0` OPTIONS {existingData: "use", seedUri:
"https://downloads.graphaware.com/neo4j-db-seeds/hmdd2.0.backup"}
```

As an example, let's try to evaluate how similar miRNAs can be, based on the number of target mRNAs they have in common. We will consider two miRNAs similar if they share many Target nodes. Figure C.19 illustrates this concept, showing how *miRNA2* is more similar to *miRNA3* than it is to *miRNA1*, with the thickness of arrow lines representing the strength of target connections.

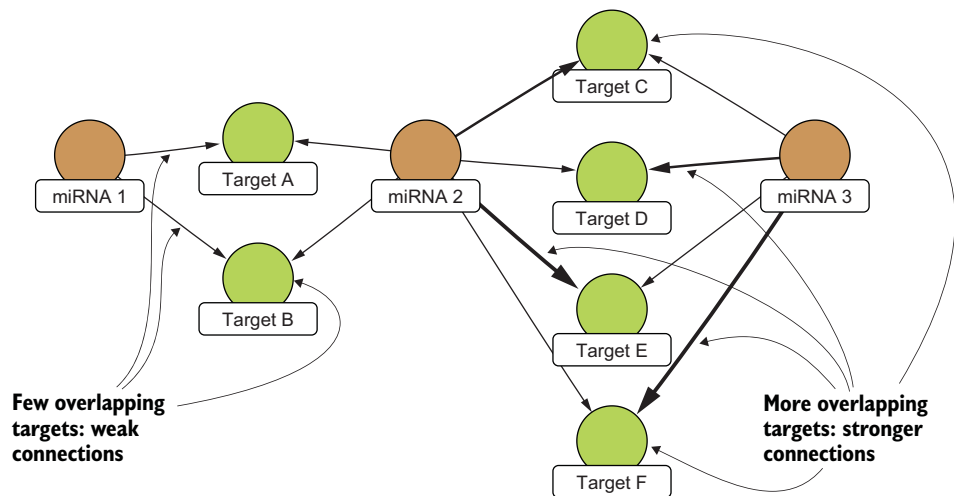


Figure C.19 *miRNA2* is more similar to *miRNA3* than it is to *miRNA1*. The thicker the arrow lines, the stronger the target's connections.

To compute this type of similarity, we will use the `nodeSimilarity` function from GDS in its weighted version. This means miRNAs connected to targets with a high score value will be considered more significant than those with weaker connections. Before using this algorithm, we have to create a named in-memory representation of the graph we are going to analyze.

Listing C.21 Creating the in-memory graph

```
CALL gds.graph.project("MiRNA_Target_similarity",
  ["Target", "MiRNA"],
  {HAS_TARGET:{properties:["value"]}})
```

← **Considers only miRNAs and Targets**

← **Includes the value property in the in-memory projection so we can use it later**

Once the in-memory database has been created, we can run the node similarity computation. The following listing calculates the similarity between nodes using the weighted version of the algorithm, allowing us to prioritize connections with higher score values.

Listing C.22 Computing similarity

```
CALL gds.nodeSimilarity.stream(
  "MiRNA_Target_similarity",
  {relationshipWeightProperty: 'value'})
YIELD node1,node2, similarity
WITH gds.util.asNode(node1) AS source,
     gds.util.asNode(node2) AS target, similarity
RETURN source.name AS source, target.name AS target, similarity
ORDER BY similarity DESC, source, target
```

← **Uses the value relationship attribute as a weight property**

The results of the node similarity computation are reported in table C.4. For the most similar miRNAs in the table, we can find some obvious (like *hsa-let-7a-5p* and *hsa-let-7c-5p* from the *let-7* family) and some less obvious examples. For example, searching on the internet for the miRNAs on line 3, *hsa-mir-107* and *hsa-mir-103s-3p*, finds many articles discussing the connection of these two miRNAs to osteoarthritis, cystic fibrosis, and other diseases.

Table C.4 Results of the similarity query in listing C.22

Source	Target	Similarity
<i>hsa-let-7a-5p</i>	<i>hsa-let-7c-5p</i>	1.0
<i>hsa-let-7a-5p</i>	<i>hsa-let-7e-5p</i>	1.0
<i>hsa-mir-107</i>	<i>hsa-mir-103a-3p</i>	1.0
<i>hsa-mir-570-5p</i>	<i>hsa-mir-548ai</i>	1.0

We can also push ourselves further. As we know, miRNAs regulate gene expression by interfering with specific mRNAs; when this regulation is abnormal, pathologies may

result. It is reasonable to wonder how a target mRNA can be considered similar, or affine, to a disease based on how many miRNAs the two have in common. In addition to being interesting in its own right, this kind of analysis demonstrates how we can compare different entities (Diseases and Targets) using information from multiple datasets.

We will again use the `nodeSimilarity` function in GDS. But this time it's a filtered version, because we are interested in similarity relations between Targets and Diseases, rather than between Targets and other Targets or between miRNAs and Diseases, for example. The following Cypher query creates the in-memory graph representation that will serve as the basis for our similarity analysis.

Listing C.23 Creating the in-memory graph

```
CALL gds.graph.project("Disease_Target_similarity",
    ["Target", "MiRNA", "Disease"],
    {HAS_TARGET:{orientation:"UNDIRECTED"},
      RELATED_TO:{orientation:"UNDIRECTED"},
      SIMILAR_TO:{orientation:"UNDIRECTED"}})
```

Makes sure we do not consider the direction of the relationships

Once the graph has been created in memory, we can run the similarity computation and evaluate the results.

Listing C.24 Computing similarity

```
CALL gds.nodeSimilarity.filtered.stream(
    "Disease_Target_similarity",
    {sourceNodeFilter:"Disease",targetNodeFilter:"Target"})
yield node1,node2, similarity
WITH gds.util.asNode(node1) AS source,
     gds.util.asNode(node2) AS target, similarity
MATCH (source)-[]-(m:MiRNA)-[:HAS_TARGET]-(target)
WITH source, target, similarity, count(m) as miRNAs
WHERE miRNAs > 10
RETURN source.name AS source, target.name AS target, similarity, miRNAs
ORDER BY similarity DESCENDING, source, target
```

Instructs the algorithm to consider only the similarity between Disease and Target node types

Counts how many miRNAs the Disease and Target have in common

Keeps only Diseases and Targets that share at least 10 miRNAs

The results of the similarity query are reported in table C.5. The query in listing C.25 examines the first row in detail; the result looks like the graph shown in figure C.20.

Table C.5 Results of the similarity query in listing C.24

Source	Target	Similarity	miRNAs
meningioma	NM_203347	0.047619048	11
meningioma	NM_001031745	0.045454545	12
prostate neoplasms	NM_012316	0.030769231	16
prostate neoplasms	NM_001260491	0.030373832	13

Listing C.25 *meningioma–NM_203347* similarity in detail

```

MATCH p=(source)-[:HAS_TARGET]-(target)-[:HAS_TARGET]-(target)
WHERE source.name = "meningioma"
AND target.name = "NM_203347"
RETURN p

```

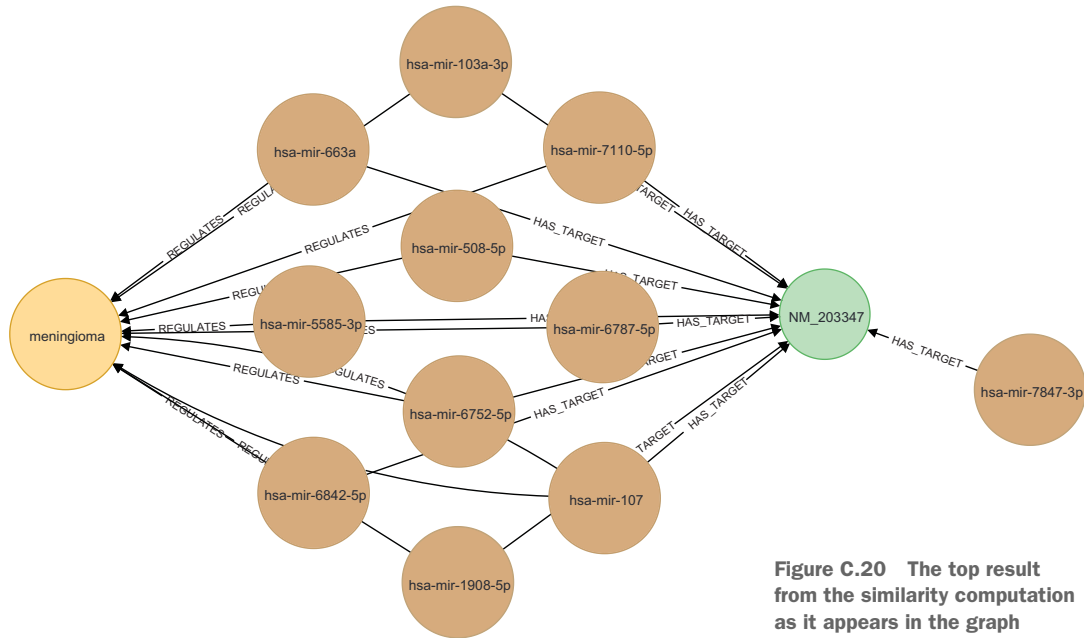


Figure C.20 The top result from the similarity computation as it appears in the graph

In this example, we can see that almost all of the miRNAs associated with the target mRNA (NM_203347) are related to meningioma. Again, these types of findings do not have to be significant from a medical standpoint, but they represent information usable by a machine learning algorithm.

The last analysis we'll run on this brand-new KG is the same one we use for Hetionet, as discussed in chapter 4, where we use degree-weighted path count (DWPC) to find relevant paths from a disease to a GO Process. As a reminder, DWPC helps us avoid biasing our analysis with nodes that are part of many paths by penalizing highly connected nodes.

In this case, we will use celiac disease as our reference disease because we are familiar with it and can effectively evaluate the results. The following listing queries our KG to identify relevant targets potentially associated with celiac disease.

Listing C.26 Searching for relevant targets connected to celiac disease

```

MATCH path = (d:Disease)-[:REGULATES|RELATED_TO]-(m)-[:HAS_TARGET]-(t)
WHERE d.name = "celiac disease"
WITH

```

```

[
  size([(d) <- [:REGULATES|RELATED_TO] - () | d]),
  size([( ) <- [:REGULATES|RELATED_TO] - (m) | m]),
  size([(m) - [:HAS_TARGET] -> ( ) | m]),
  size([( ) - [:HAS_TARGET] -> (t) | t])
]
AS degrees, path, d, t
WITH d.name as disease_name, t.name as target_name, count(path) as PC,
sum(reduce(pdp = 1.0, d in degrees | pdp * d ^ -0.4)) AS DWPC,
size([(t) - [:HAS_TARGET] - ( ) | t]) AS n_miRNA
WHERE n_miRNA >= 5 and PC >= 2
RETURN disease_name, target_name, PC, DWPC, n_miRNA
ORDER BY DWPC desc
LIMIT 10

```

The results are summarized in table C.6. They're interesting because they align with the scientific evidence related to celiac disease. For example, the first target in the table is *NM_080601*: "homo sapiens protein tyrosine phosphatase non-receptor type 11 (PTPN11), transcript variant 2, mRNA." Research has shown the role of protein tyrosine phosphatases in regulating the immune system and the implications this has for chronic intestinal inflammation [34].

Targets 2, 3, and 5 in the table (*NM_001224*, *NM_032982*, and *NM_032983*) are all variants of "homo sapiens caspase 2, apoptosis-related cysteine peptidase," *CASP2*. Several studies, including a single-cell RNA-seq survey of gluten-specific T cells [35], provide the knowledge base for finding unique targets for the removal of gluten-specific T cells as a curative therapeutic option for celiac disease. In these studies, researchers found marked upregulation of several apoptosis-related genes, such as *FAS*, *TRAIL*, and *CASP2*, in tetramer-positive cells, possibly due to in vivo activation by gluten antigens. These findings encourage the use of activation-induced cell death for the removal of gluten-specific T cells.

Table C.6 Results of the query in listing C.26

Disease	Target	PC	DWPC	# miRNA
celiac disease	NM_080601	2	0.00417	25
celiac disease	NM_001224	2	0.00322	111
celiac disease	NM_032982	2	0.00318	114
celiac disease	NM_152617	2	0.00295	136
celiac disease	NM_032983	2	0.00278	160
celiac disease	NM_198926	3	0.00241	158
celiac disease	NM_019099	3	0.00234	169
celiac disease	NM_005235	2	0.00219	286
celiac disease	NM_052845	2	0.00210	138
celiac disease	NM_001142551	2	0.00209	138

We didn't find any easy-to-access articles connecting celiac disease to related gene variants expressed by the other targets in the table. But considering the active research surrounding this disease, it is possible that a correlation will become apparent.

To complete our analysis, let's run the query showing how miRNAs are connected to the first target in the list. The next listing searches for all paths connecting celiac disease with the target *NM:080601*; the result is shown in figure C.21.

Listing C.27 Searching for all the paths between *celiac disease* and *NM_080601*

```
MATCH path = (d:Disease) <- [:REGULATES|RELATED_TO] - (m) - [:HAS_TARGET] -> (t)
WHERE d.name = 'celiac disease'
      AND t.name = 'NM_080601'
RETURN path
```

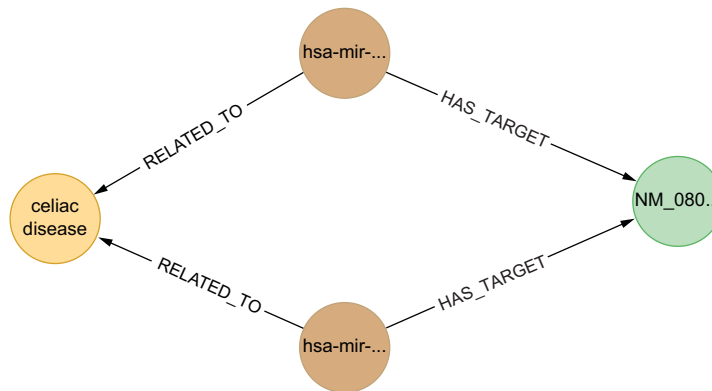


Figure C.21 Paths between *celiac disease* and the target *NM_080601*, which is first in the results

As we've demonstrated, combining multiple sources in a single, holistic KG enables us to analyze information from various perspectives. In addition, metrics like DWPC have broad applications across contexts.

Exercise

If you are interested in the domain, run some of the previous queries again, changing the target disease, and evaluate the results.